

THE ZERO PARADOX

ZP Addendum

The Choice-Free Core

Initial: June 2026 | Current: 1.8, August 2026

Build the file `ZeroParadox/AxiomProfile.lean` and read the Lean kernel's output. It reports that the central theorem of this framework — the Binary Snap, T-SNAP, whose SHAPE is derived while its occurrence stays a framework commitment, the transition $\perp \rightarrow \varepsilon_0$ — depends on no axioms at all: not the Axiom of Choice, not even propositional extensionality. The lattice algebra (ZP-A) and the Quine-atom self-reference that is the framework's keystone (ZP-J) are likewise choice-free. This is a machine-checked fact, not a claim of the prose: anyone can run ``lake build ZeroParadox.AxiomProfile`` and see it.

Two boundaries are stated up front, because the claim is narrow and exact. The framework as a whole is not choice-free. Most of its theorems do depend on ``Classical.choice``. Most places it appears are places where the framework builds on Mathlib's classically-built analysis, order, and computability libraries — the layers that realize the snap inside standard analytic structures (p-adic topology, Hilbert space, ordinals), where the dependence is inherited from those libraries. The category-theory face is the exception: its choice is the framework's own bare classical, and Section III shows it essential rather than inherited. It is not used by the core results above. And dependence is not necessity: that those realizations use choice as written does not show choice is required there (Section III).

Section I: The Choice-Free Core

The Lean kernel's ``#print axioms`` command reports the complete axiom dependency of any result. Run on the framework's central results, it returns the following. "Does not depend on any axioms" is the strongest possible report — stronger than "choice-free," since it uses not even propositional extensionality.

Verified axiom-free (does not depend on any axioms)

T-SNAP, the Binary Snap and its derivation (ZP-E):

`t_snap_machine`, `t_snap_derived`, `t_snap_join`, `t_snap_irreversible`,

`da1_minimal_path`, `dp2_execution_distinguishability`.

The lattice algebra (ZP-A): `bot_le`, the order laws, `cc1`.

The Quine-atom self-reference keystone (ZP-J): `bot_is_quine_atom`, `cc1_derived`,

`t_exec`, `quine_atom_unique`.

A second tier of results is choice-free but uses propositional extensionality and quotient soundness (`[propext, Quot.sound]`), both standard in Lean 4. These include the structural floor (`PowerSet.ps_structural_floor`) and the wheel of fractions (`WheelFrac.instWheel, inf_ne_bot`). No `Classical.choice`.

Section II: Where Classical.choice Enters

The honest contrast. `Classical.choice` does appear across the framework — in the majority of its theorems — and the same `#print axioms` artifact shows exactly where. Most occurrences are in a layer that realizes the snap floor inside a standard analytic structure, inheriting choice from the Mathlib library that builds that structure classically. The category-theory face is not one of them — see Section III.

Carries Classical.choice (inherited from Mathlib), e.g.
ZP-B <code>c3_irreversible</code> — p-adic topology (metric / ultrametric library).
ZP-D <code>t4_snap_orthogonal</code> — Hilbert space (inner-product library).
ZP-H <code>fB_functor</code> / <code>fD_functor</code> / <code>fC_functor</code> — TopCat / ModuleCat $\mathbb{C}$ /
the Kleisli category of the probability monad.
and the ordinal (ZP-L/M), information (ZP-C), and computability (ZP-K) layers.
Footprint in each case: <code>[propext, Classical.choice, Quot.sound]</code> .

The pattern is clean: the core states the result; the analytic layers realize it inside the standard frameworks, and that is where the library's classical foundations enter. For those layers the choice is in the plumbing, not in the claim — but not for all of it: Section III locates two principles that are provably essential. Their provenance differs, and that is the point: one spends the framework's own bare classical, the other spends Mathlib's — and both are essential. Inherited never meant removable.

Section III: Dependence Is Not Necessity

`#print axioms` proves dependence — that the proof as written uses an axiom. It does not prove necessity — that no choice-free proof exists. Necessity has to be shown a different way, by a reduction: derive a recognised constructive taboo from the principle itself.

Two such reductions do exist here, and they bound what this document claims.

`em_of_wellOrder_comparable` derives excluded middle from comparability of well-orders (prior art: Kraus, Nordvall Forsberg and Xu, arXiv:2104.02549, Thm 38(d)), and `wem_of_fixedPointFree` derives weak excluded middle from the general fixed-point-free principle — the latter sitting on the keystone rather than on an imported order instance. Each is a statement about the principle: re-proving it constructively would decide a taboo, so no choice-free re-proof exists. Neither is an independence result, and neither is established by a footprint measurement — which is precisely why the accidental side needs a measurement and the essential side needs a reduction.

So the analytic-layer dependence may be removable, and that is a different question from the two cases above. It is also not settled by measuring a proof: where an axiom sits in a type rather than a proof, no proof of any statement mentioning that type can be clean, and removability there means changing the statement, not cleaning the argument.

One layer has been classified directly. In the "choice-probe" experiment, the ``Classical.choice`` in the 2-adic tree construction (PadicTree) decomposed into three sources: incidental tactic artifacts (removed, leaving those results choice-free); Mathlib's classically-proved connectivity API (routable by a path-uniqueness reformulation); and ``sInf`` on a complete lattice (routable by a redefinition). The verdict for that layer was "mostly not structurally required." Whether this generalizes — and whether the snap geometry forces choice anywhere — is an open question, tracked for the constructive validation layer (ONote/NOntNote, future ZP-N).

Remark — Why this matters

The Zero Paradox argues that the foundational axioms are not freely chosen but forced by the structure of the bottom element. It would be a tension if the framework's own central results leaned on the Axiom of Choice — the canonical free, non-constructive selection. They do not. T-SNAP is axiom-free; the keystone is choice-free. The "forced, not chosen" thesis is internally consistent at the level of what the framework actually asserts. Where choice appears it is mostly the supporting library's classical foundation showing through the realizations rather than an assumption of the argument. Section III's two cases are the exception on a different axis: they are about necessity, not provenance. One of them is the framework's own and one is Mathlib's, and both are essential — so "inherited" is not a synonym for "removable", and neither case is absorbed into the generalization.

The Artifact

Checkable evidence

ZeroParadox/AxiomProfile.lean — a file of ``#print axioms`` commands. Section I prints the choice-free core; Section II prints the analytic-realization results that carry choice, as an honest contrast. Build with ``lake build ZeroParadox.AxiomProfile`` and read the kernel's report. The per-theorem map is in the project repository.

Choice-probe (the one-layer classification): branch ``choice-probe``; the verdict and the three decomposed sources are recorded in the project notes.

Scope of the claim

The claim is exactly: the framework's central results — T-SNAP, the lattice, the Quine-atom self-reference — are choice-free, and T-SNAP is axiom-free. NOT claimed: that the whole framework is choice-free (it is not), nor that the analytic-layer choice is removable or necessary (open). The fact surfaced here is the verified one.

Endnote: This is a framework-wide note, machine-verified as of June 2026. The central theorem T-SNAP depends on no axioms; the conceptual core is free of the Axiom of Choice; ``Classical.choice`` appears mostly where the framework builds on Mathlib's classical analysis, order, and computability libraries, where whether it is necessary remains open. It is not only there, and not everywhere open: the category-theory face carries the framework's own bare classical, and Section III's two reductions settle necessity for the principles they cover. All of this is checkable in ZeroParadox/AxiomProfile.lean.